

Hardware-Adaptive Numerics for MFEM through Compiler-Based Rewriting



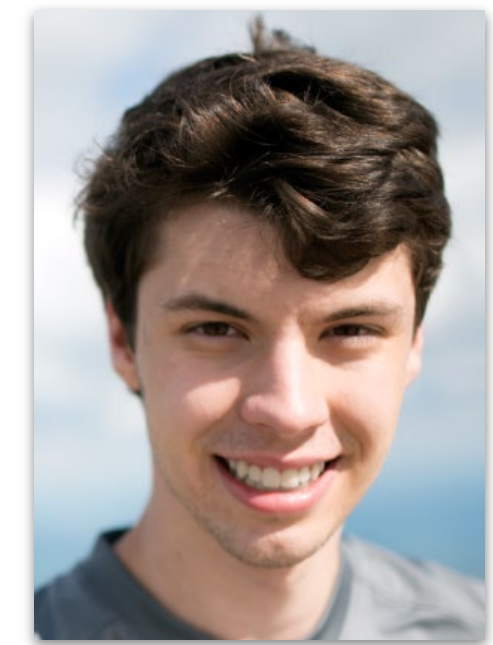
Siyuan Brant Qian¹



Julian Andrej²



Tzanio Kolev²



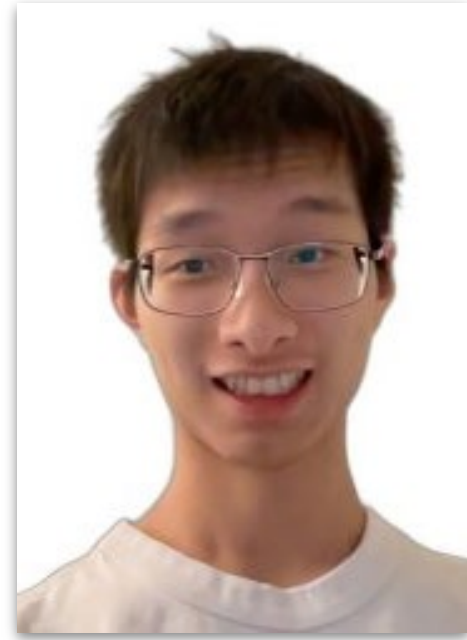
William S. Moses¹

siyuanq4@illinois.edu

¹University of Illinois Urbana-Champaign, USA

²Lawrence Livermore National Laboratory, USA

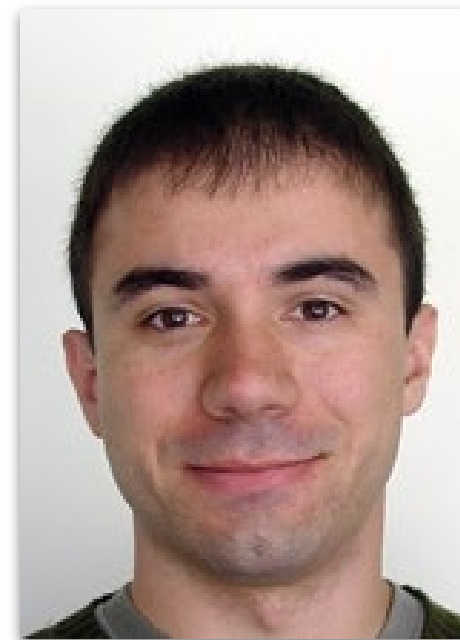
MFEM Community Workshop 2026, Atlanta, GA
September 23, 2026



Siyuan Brant Qian



Julian Andrej



Tzanio Kolev



Vimarsh Sathia



Ivan R. Ivanov



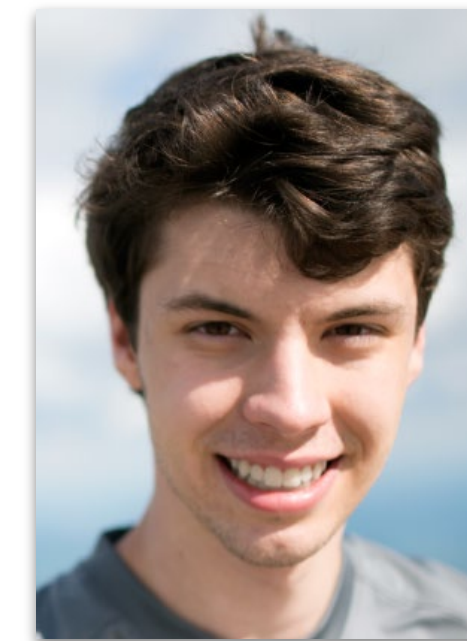
Jens Domke



Jan Hückelheim



Paul Hovland



William S. Moses

Finite Elements at Extreme Scale

- Finite element applications increasingly demand computation at **extreme scale** and predictions in **real time**.
- A digital twin* for tsunami early warning used **55.5 trillion unknowns across 43,520 GPUs** and demonstrated **billion-parameter inference in 0.2 seconds**.

Geophysics

Solid Mechanics

Fluid Dynamics

Cardiac Modeling

Image sources: <https://dl.acm.org/doi/10.1145/3712285.3771787> (Geophysics) and <https://mfem.org/gallery/> (Solid Mechanics, Fluid Dynamics, Cardiac Modeling)

3 * Henneking et al., Real-Time Bayesian Inference at Extreme Scale: A Digital Twin for Tsunami Early Warning Applied to the Cascadia Subduction Zone (SC25; 2025 ACM Gordon Bell Prize).



Accelerator Numerics: The Throughput Challenge

- Accelerators supply much of the compute for large-scale analysis
- However, writing fast accelerator code is challenging:
 - **FP64 is increasingly deprioritized in favor of lower-precision arithmetic**, but naively switching to lower precision can produce **incorrect** results
 - Most of the peak arithmetic throughput lies in **specialized hardware units** that perform only specific operations



Accelerator Numerics: The Throughput Challenge

- Accelerators supply much of the compute for large-scale analysis
- However, writing *fast* accelerator code is challenging:
 - **FP64** is increasingly **deprioritized**
 - Most of the peak arithmetic throughput lies in **specialized hardware units** that perform only specific operations

Hopper (GH200)

RTX Blackwell (RTX 5090)

Blackwell Ultra (B300)

Rubin NVL8 (preliminary)

FP64 Scalar

FP32 Scalar

INT8 Tensor Cores

FP64 Tensor Cores

FP16 Tensor Cores

x-axes: trillions of operations per second
Ratios: relative throughput to native FP64 on the same device
RTX 5090 FP64 Tensor Cores shown schematically; minimal hardware for correctness.



Writing fast and accurate numerical code is a challenge of
context, scope, and scale.



Hardware Context: Strategies Differ across Devices

MFEM parallel loop

Portable baseline

```
// C = A B; M = N = K = 4096
mfem::forall(M, N,
[=] MFEM_HOST_DEVICE(int i, int j) {
    double sum = 0.0;
    for (int k = 0; k < K; ++k)
        sum += A[i + k*M] * B[k + j*K];
    C[i + j*M] = sum;
});
```



Hardware Context: Strategies Differ across Devices

MFEM parallel loop

Portable baseline

```
// C = A B; M = N = K = 4096
mfem::forall(M, N,
 [=] MFEM_HOST_DEVICE(int i, int j) {
   double sum = 0.0;
   for (int k = 0; k < K; ++k)
     sum += A[i + k*M] * B[k + j*K];
   C[i + j*M] = sum;
 });
```

FP64 Tensor Cores

cuBLAS DGEMM

```
const double alpha = 1.0;
const double beta = 0.0;
cublasDgemm(handle,
  CUBLAS_OP_N, CUBLAS_OP_N,
  M, N, K,
  &alpha, A, M,
  B, K,
  &beta, C, M);
```

20.3× faster on GH200

No performance gain on RTX 5090



Hardware Context: Strategies Differ across Devices

MFEM parallel loop

Portable baseline

```
// C = A B; M = N = K = 4096
mfem::forall(M, N,
[=] MFEM_HOST_DEVICE(int i, int j) {
    double sum = 0.0;
    for (int k = 0; k < K; ++k)
        sum += A[i + k*M] * B[k + j*K];
    C[i + j*M] = sum;
});
```

FP64 Tensor Cores

cuBLAS DGEMM

```
const double alpha = 1.0;
const double beta = 0.0;
cublasDgemm(handle,
    CUBLAS_OP_N, CUBLAS_OP_N,
    M, N, K,
    &alpha, A, M,
    B, K,
    &beta, C, M);
```

20.3× faster on GH200

No performance gain on RTX 5090

INT8 emulation

Ozaki-II Scheme*

```
const double *X = B, *Y = A;
poz_absMax<<<grid, tile>>>(
    X, Y, maxX, maxY, N);
poz_shifts<<<blocks, 128>>>(
    maxX, maxY, sx, sy, N, beta_exp);
poz_fillRes<<<grid, tile>>>(
    X, Y, sx, sy, Xr, Yr, N, Np);
cublasGemmStridedBatchedEx(h,
    CUBLAS_OP_T, CUBLAS_OP_N,
    Np, Np, Np,
    &i_one, Yr, CUDA_R_8I, Np, plane,
    Xr, CUDA_R_8I, Np, plane,
    &i_zero, Cr, CUDA_R_32I, Np, plane,
    13, CUBLAS_COMPUTE_32I,
    CUBLAS_GEMM_DEFAULT);
poz_crt<<<grid2, dim3(32, 8)>>>(
    Cr, sx, sy, C, N, Np, 1.0, 0.0);
```

17.7× faster on RTX 5090;

relative error $\leq 10^{-12}$

* K. Ozaki, Y. Uchino, and T. Imamura. *Ozaki Scheme II: A GEMM-oriented emulation of floating-point matrix multiplication using an integer modular technique.*



Numerical Context: Accuracy Requirements Vary

```
double sum = 0.0;
for (double x : xs) {
    double val = sigmoid(exp(x));
    sum += val;
}
```



Numerical Context: Accuracy Requirements Vary

```
double sum = 0.0;
for (double x : xs) {
    double val = sigmoid(exp(x));
    sum += val;
}
```

```
double sum = 0.0;
for (double x : xs) {
    float val = sigmoidf(expf(x));
    sum += val;
}
```

Lower Activation:
1.6× speedup
Relative error $\approx 1e-12$



Numerical Context: Accuracy Requirements Vary

```
double sum = 0.0;
for (double x : xs) {
    double val = sigmoid(exp(x));
    sum += val;
}
```

```
double sum = 0.0;
for (double x : xs) {
    float val = sigmoidf(expf(x));
    sum += val;
}
```

```
float sum = 0.0f;
for (double x : xs) {
    float val = sigmoidf(expf(x));
    sum += val;
}
```

Lower Activation:
1.6× speedup
Relative error $\approx 1e-12$

Lower Reduction:
Relative error $\approx 1e-4$



Scope: Beyond Precision Changes

FP32 (Original Order)

⚡ Fast

✗ Wrong

$(1e8 + 1) - 1e8 \rightarrow 1.00000000e8 - 1e8 \rightarrow 0$

+1 rounded away



Scope: Beyond Precision Changes

FP32 (Original Order)

$(1e8 + 1) - 1e8 \rightarrow 1.00000000e8 - 1e8 \rightarrow 0$
+1 rounded away

⚡ Fast

✗ Wrong

FP64 (Same Order)

$(1e8 + 1) - 1e8 \rightarrow 1.000000001e8 - 1e8 \rightarrow 1$

🐢 Costly

✓ Correct

Use FP64
(Tradeoff!)



Scope: Beyond Precision Changes

FP32 (Original Order)

$(1e8 + 1) - 1e8 \rightarrow 1.00000000e8 - 1e8 \rightarrow 0$

+1 rounded away

⚡ Fast

✗ Wrong

FP64 (Same Order)

$(1e8 + 1) - 1e8 \rightarrow 1.000000001e8 - 1e8 \rightarrow 1$

🐢 Costly

✓ Correct

Use FP64
(Tradeoff!)

FP32 (Reassociated)

$(1e8 - 1e8) + 1 \rightarrow 0 + 1 \rightarrow 1$

⚡ Fast

✓ Correct

Algebraic Rewrite
(circumvents tradeoff
but requires context)



Scale: Numerical Optimization in Practical Applications

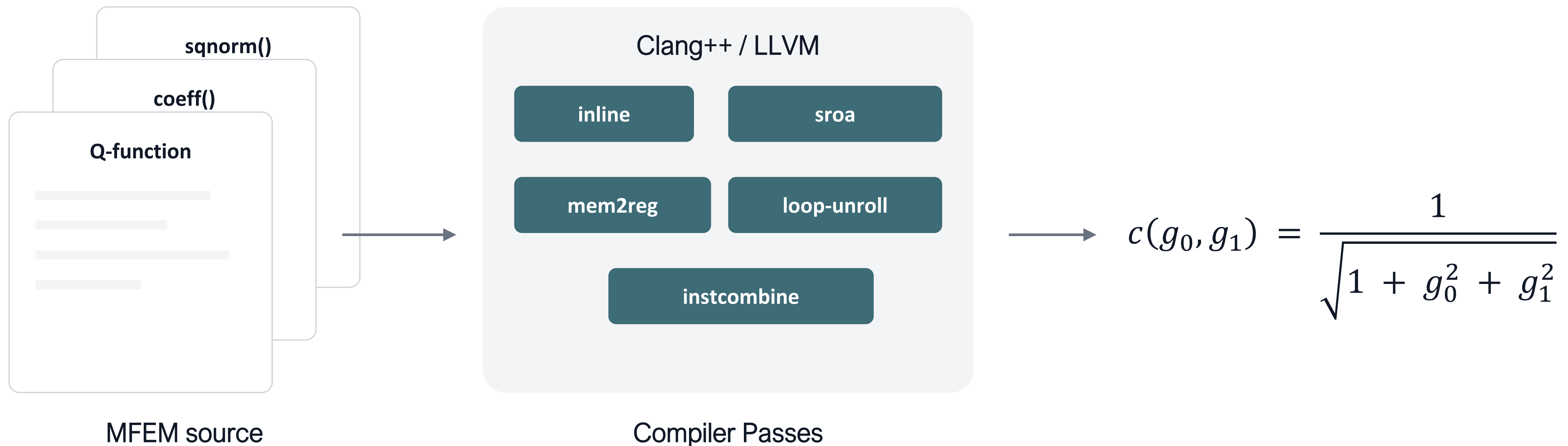
```
const auto invJ = inv(J);  
const auto dudx = dudxi * invJ;  
return tuple{  
    coeff(dudx)  
    * dudx * transpose(invJ)  
    * det(J) * w  
};
```

```
auto coeff(const tensor<T, dim>& a) {  
    return 1_r / sqrt(1_r + sqnorm(a));  
}
```

```
T sqnorm(const tensor<T, m>& a) {  
    T total{};  
    for (int i = 0; i < m; ++i)  
        total += a[i] * a[i];  
    return total;  
}
```



Scale: Numerical Optimization in Practical Applications



Compiler analyses and optimizations expose the underlying math!



Can we build a finite element library that automatically **compiles**
user-defined physics into **fast code for each device**
while meeting **required accuracy thresholds**?



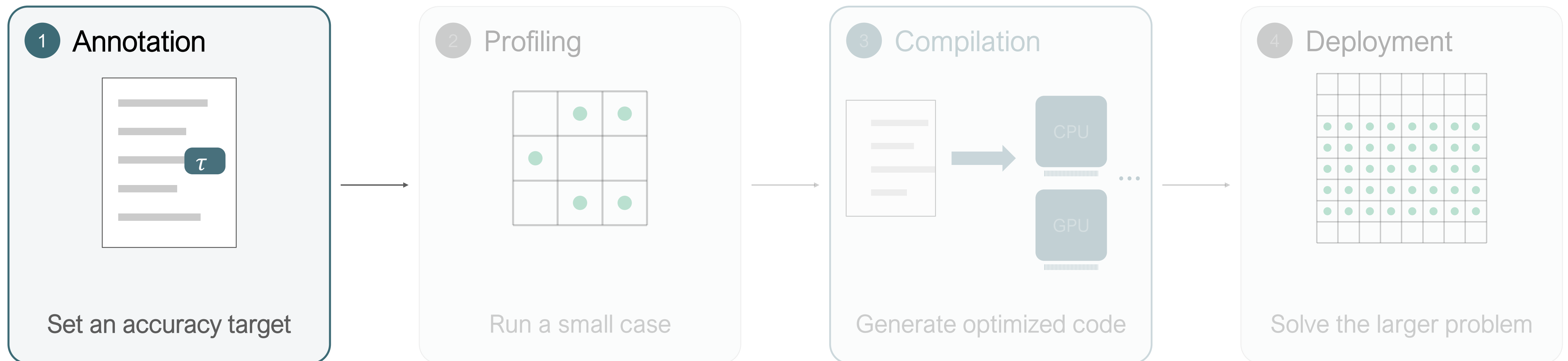
MFEM-Poseidon: Hardware-Adaptive Numerics for MFEM

- Poseidon is a Clang/LLVM compiler plugin that optimizes numerical code under an accuracy target with profiles and a device cost model (calibrated once per device).



MFEM-Poseidon: Hardware-Adaptive Numerics for MFEM

- Poseidon is a Clang/LLVM compiler plugin that optimizes numerical code under an accuracy target with profiles and a device cost model (calibrated once per device).
- MFEM-Poseidon integration enables automated numerical rewriting to operators, custom physics, and generated derivatives.



MFEM Integration: Annotating Integrators and Function Calls

- Specify an accuracy target τ for an integrator or a function call
- Poseidon selects numerical rewrites using an error estimate based on profiled data, subject to τ .

1 Integrators

```
dop.AddDomainIntegrator  
  <LocalQFBackend>(  
    qf, inputs, outputs, *ir, attr,  
    Numerics<Tolerance<1e-9>>{}));
```

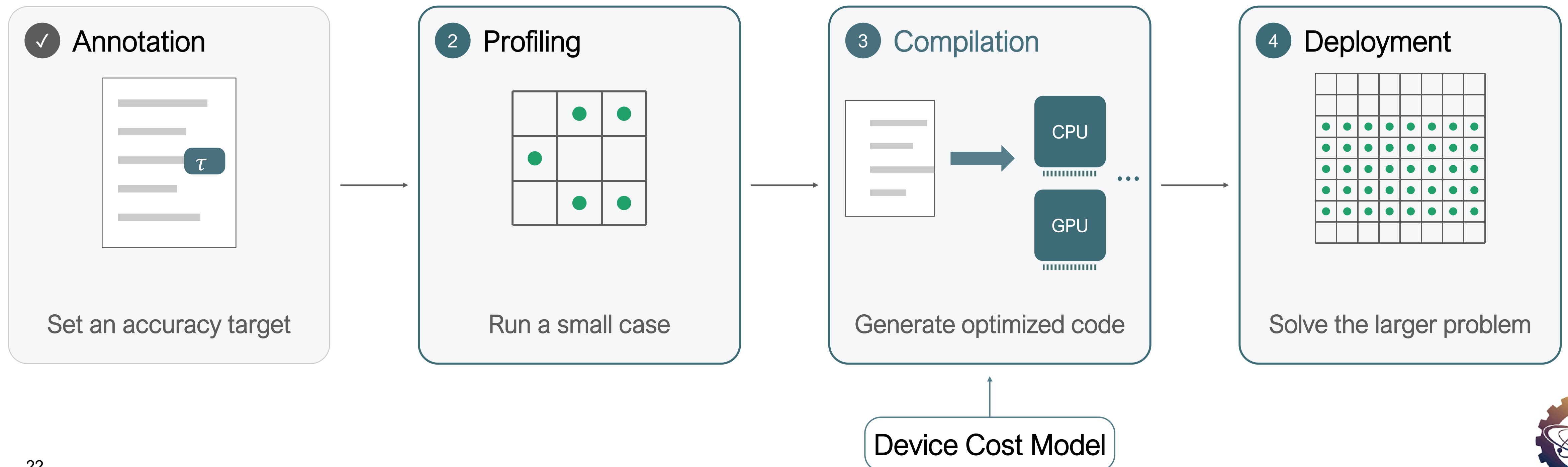
2 Function calls

```
__poseidon_fp_optimize<void>(  
  (void *)pressure, poseidon_tau, 1e-6,  
  dudx, &p);
```



MFEM-Poseidon: Hardware-Adaptive Numerics for MFEM

- Poseidon optimizes numerical code under an accuracy target with profiles and a device cost model (calibrated once per device).
- MFEM-Poseidon integration enables automated numerical rewriting to operators, custom physics, and generated derivatives.



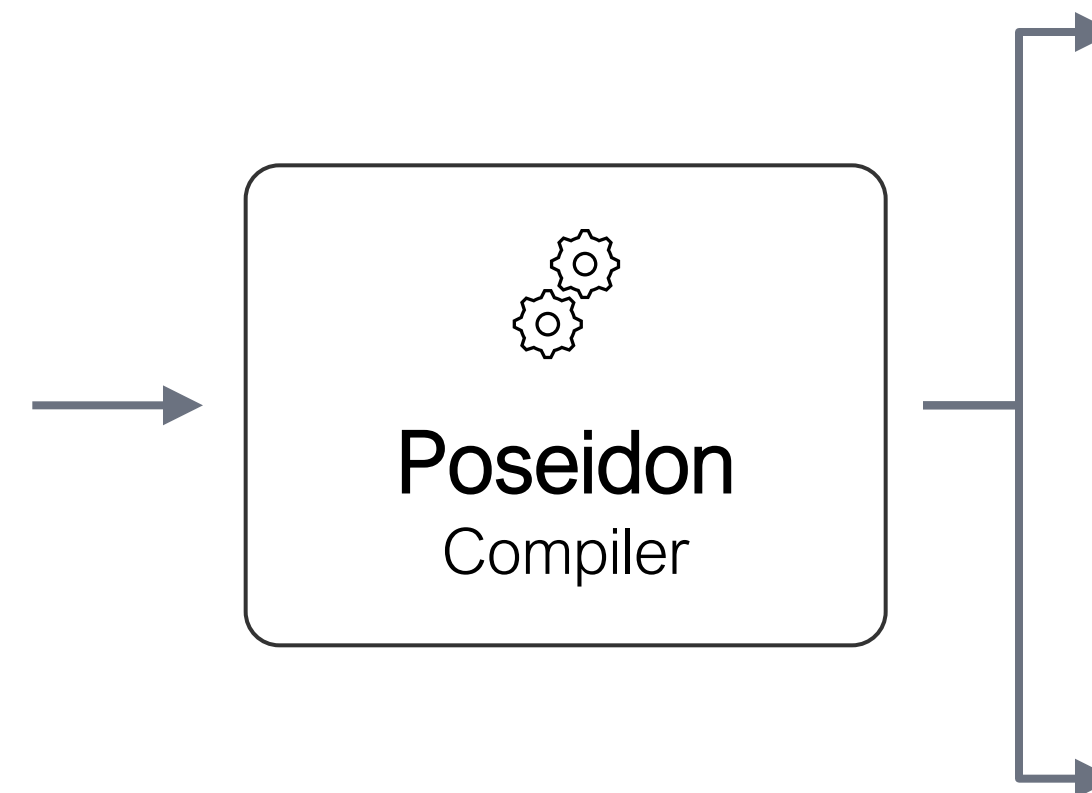
Elasticity Operator: 9.6× Faster at Near-FP64 Accuracy

- Different accuracy targets τ automatically enable different optimizations.
- At $\tau = 10^{-12}$, an emulation rewrite delivers **9.6× faster** operator evaluation.
- **5.05× faster CG solve**, with the same 1398 iterations.
- Looser targets permit reduced-precision GEMM.



Elasticity Operator: Hardware-Adaptive Numerics

```
void ElasticityIntegrator::  
  AddMultPA(const Vector &x,  
            Vector &y) const  
{  
  __poseidon_fp_optimize<void>(  
    (void *)internal::  
      ElasticityAddMultPA,  
      poseidon_tau, tau,  
      vdim, ndofs, *fespace,  
      *lambda_quad, *mu_quad,  
      *geom, *maps, x,  
      *q_vec, y);  
}
```



GH200
FP64 Tensor Cores
22.5× faster operator evaluation

RTX 5090
INT8 tensor-core-based emulation
9.14× faster operator evaluation



Diffusion Operator: Near-FP64 Accuracy with FP32 Expansion

- Represent each value with two FP32 components.
- Despite the increased operation count, FP32 expansion runs faster on RTX 5090, with **~4× lower** estimated arithmetic cost.
- **1.57× faster** operator evaluation, with relative error 4.3×10^{-15} .

```
// Original FP64 inner-loop update  
u += coords * b;
```

Automatically rewritten to...

```
// Each value becomes a pair of FP32 numbers  
float p = coords.hi * b.hi;  
float e = fmaf(coords.hi, b.hi, -p);  
e = fmaf(coords.hi, b.lo, e);  
e = fmaf(coords.lo, b.hi, e);  
float s = p + u.hi;  
float pv = s - u.hi;  
float uv = s - pv;  
float r = (p - pv) + (u.hi - uv);  
float t = e + u.lo + r;  
u.hi = s + t;  
u.lo = t - (u.hi - s);
```



Hooke Miniapp: Avoiding Cancellation

```
real_t J = det(I + dux);  
real_t p = -2.0 * D1 * J  
          * (J - 1);  
auto devB = dev(dux  
  + transpose(dux)  
  + dot(dux, transpose(dux)));  
auto sigma = -(p / J) * I  
  + 2 * (C1 / pow(J, 5.0 / 3.0))  
  * devB;
```

Profiled: $J - 1 \in [1.05 \times 10^{-4}, 1.80 \times 10^{-3}]$

In FP32, cancellation reduces the accuracy of $(J - 1)$ to only 3 to 4 significant digits!



Hooke Miniapp: Avoiding Cancellation

```
real_t J = det(I + dux);
real_t p = -2.0 * D1 * J
           * (J - 1);
auto devB = dev(dux
  + transpose(dux)
  + dot(dux, transpose(dux)));
auto sigma = -(p / J) * I
  + 2 * (C1 / pow(J, 5.0 / 3.0))
  * devB;
```

Profiled: $J - 1 \in [1.05 \times 10^{-4}, 1.80 \times 10^{-3}]$
In FP32, cancellation reduces the accuracy of $(J - 1)$ to only 3 to 4 significant digits!

Underlying math exposed by compiler optimizations

$$a_{ij} = (\nabla u)_{ij}$$

$$J = (1 + a_{00})(1 + a_{11})(1 + a_{22}) - (1 + a_{00})a_{12}a_{21} \\ - a_{01}a_{10}(1 + a_{22}) - a_{02}(1 + a_{11})a_{20} \\ + a_{01}a_{12}a_{20} + a_{02}a_{10}a_{21}$$

$$p = -2D_1 \cdot J \cdot (J - 1)$$

$$\sigma_{vol} = -p / J$$

Automatically rewritten as...

$$\sigma_{vol} = 2D_1 [a_{22} + a_{00}(1 + a_{22}) + a_{11}(1 + a_{00})(1 + a_{22}) \\ - (1 + a_{00})a_{12}a_{21} - a_{01}a_{10}(1 + a_{22}) - a_{02}(1 + a_{11})a_{20} \\ + a_{01}a_{12}a_{20} + a_{02}a_{10}a_{21}]$$

Eliminates cancellation from $(J - 1)$!



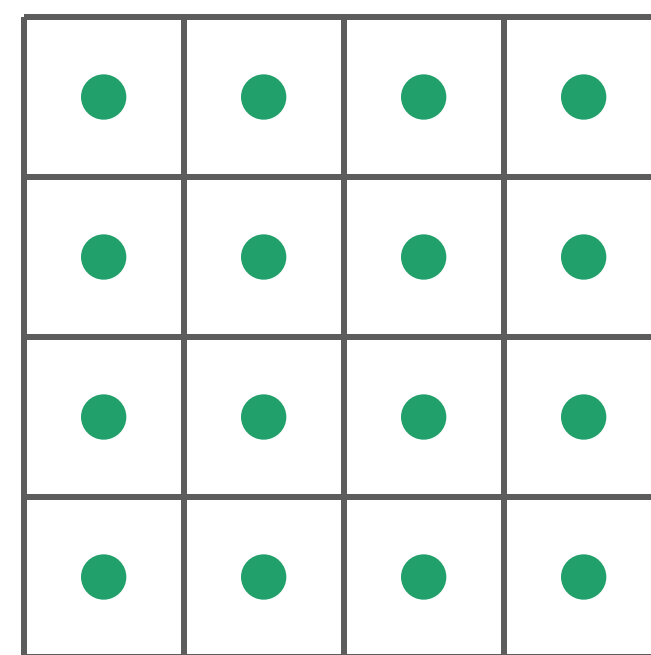
Hooke Miniapp: Poseidon-Rewritten FP32 Converges Like FP64

- **Naively switching to FP32 without changing the formula** causes the solver to stall above tolerance.
- Rewritten FP32 converges in 6 Newton steps, matching FP64.
- **1.66× end-to-end speedup** over FP64.

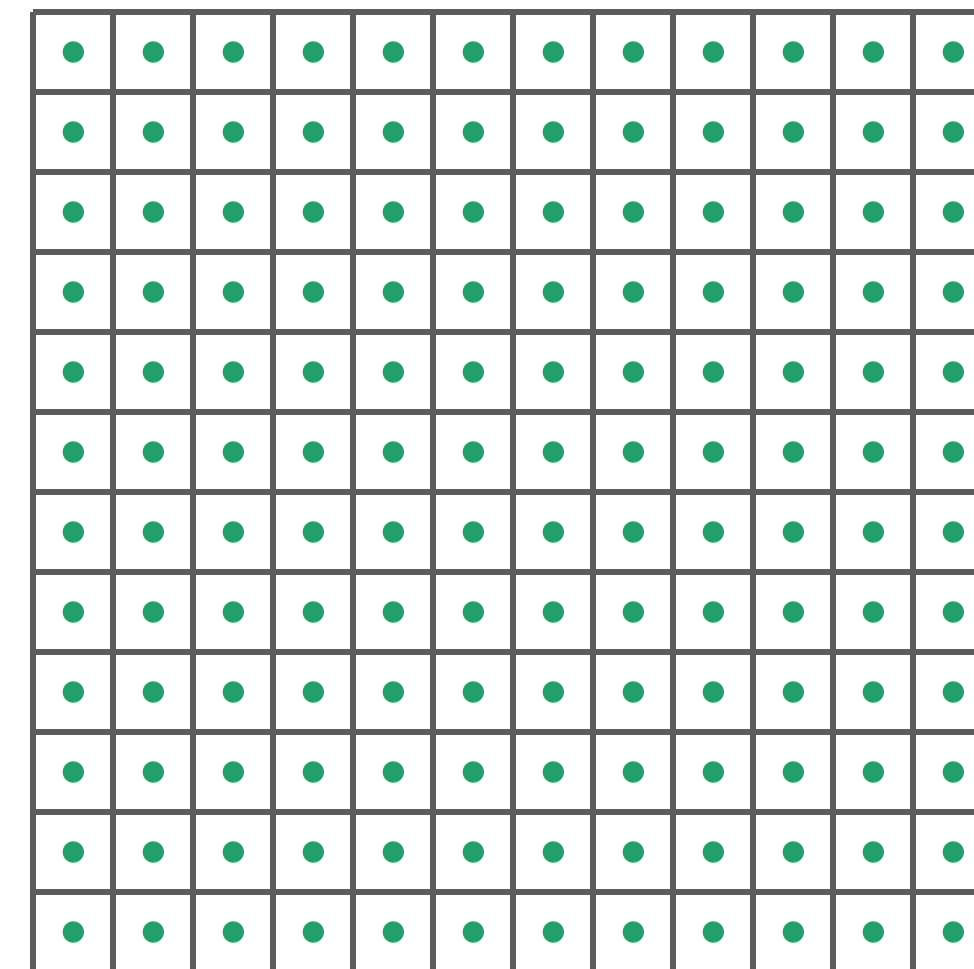


Profile Reuse across Mesh Sizes

- For the diffusion operator, the rewrite retained near-FP64 accuracy on a mesh with **512× more** elements than the profiled mesh.
- Trying different accuracy targets is cheap: recompilation **reuses** the profile and cached optimization results.



Same optimized code



Profile on a small problem

Run on a larger problem



Extending Numerical Optimization across MFEM

- Rewrites also apply to user-defined dFEM q-functions and AD-generated derivatives.
- Accuracy targets can be specified separately for residual and Jacobian evaluation.
- Minimal-surface example: **2.0×** faster Jacobian action, with the same iteration counts.

User-defined dFEM q-function

Residual: FP64

```
struct MFAppl
{
    auto operator()(const tensor<dscalar_t, dim> &dudxi,
                    const tensor<real_t, dim, dim> &J,
                    const real_t &w,
                    tensor<dscalar_t, dim> &dvdxd) const
    {
        const auto invJ = inv(J);
        const auto dudx = dudxi * invJ;
        dvdxd = coeff(dudx) * dudx * transpose(invJ) * det(J) * w;
    }
};

auto derivatives = std::integer_sequence<size_t, SOLUTION_U> {};
res->AddDomainIntegrator<LocalQFBackend>{
    mf_apply_qf, input_operators, output_operators,
    ir, all_domain_attr, derivatives,
    Numerics<DerivativeTolerance<1e-6>> {}>};
```

Enzyme AD
→

AD-generated Jacobian action

Jacobian: FP32

```
struct ManualDerivativeApply
{
    auto operator()(const tensor<real_t, dim> &dudxi,
                    const tensor<real_t, dim> &dudxi,
                    const tensor<real_t, dim, dim> &J,
                    const real_t &w,
                    tensor<real_t, dim> &dvdxd) const
    {
        const auto invJ = inv(J);
        const auto dudx = dudxi * invJ;
        const auto ddelta_udx = ddelta_udxi * invJ;
        const auto c = coeff(dudx);
        const auto term1 = c * ddelta_udx;
        const auto term2 = c * c * c * dot(dudx, ddelta_udx) * dudx;
        dvdxd = (term1 - term2) * transpose(invJ) * det(J) * w;
    }
};
```





MFEM-Poseidon

- Write operators and custom physics in MFEM, and specify the required accuracy.
- MFEM-Poseidon adapts **precisions, algebraic forms, and hardware instructions** to the target device.
- **Faster operator evaluation and application runs** while maintaining accuracy!

